Finite-Domain Quantified Nullivance Logic: A Machine-Checked Completeness Theorem for a Macro-Free Extensional Tableau*

Trinh Tung Lam†

9 November 2026
Part III: Finite-Domain Quantified Logic — DR0149-r3

Abstract

We extend nullivance propositional logic (NPL) — a paraconsistent, two-channel logic on the unit square with a harmonization connective $\oplus$ and four threshold signs T^+, T^-, F^+, F^- — by finite-domain quantifiers. The language has no function symbols and uses a fixed signature with crisp equality; models interpret predicates over a fixed finite nonempty domain by values in the four-element matrix **4**, and the finite universal (existential) quantifier takes the pointwise minimum (maximum) on the truth channel and the dual on the falsity channel. Main results, each machine-checked in Lean 4: (i) the exact threshold projection of the propositional theory lifts to the quantified language over finite domains; (ii) four tempting proof routes are *refuted* by explicit countermodels: the naive lifted tableau (crisp $x{=}x$ is valid but underivable), the bare grounding reduction to propositional consequence, and a dispatcher shortcut and the head-sensitive certificate bridge in the simulation program; (iii) the repaired *core extensional tableau*, whose closure conditions add four crisp-equality clauses and two ground-closure clauses to the decomposition rules, is sound and *complete*: a branch closes exactly when no finite model satisfies it. The completeness proof is a quantified tableau engine driven by a domain-weighted measure ($\mathrm{qsize}(\forall x\,\varphi) = (n{+}1)\cdot\mathrm{qsize}(\varphi)+1$ over a domain of size $n{+}1$) under which every rule strictly decreases branch weight, together with a canonical-model construction at the literal stage keyed by an injective ground-atom encoding. As a corollary, closure of the grounded propositional branch under rigid constraints reaches the core calculus with no propositional-simulation macro — settling the semantic bridge. We also document the failure analysis that led from a derivation-by-derivation simulation program to the weighted-measure proof, and a conservative membership-selecting repair of the replay certificate. The repair is sound, passes the motivating regression, and is complete for every admissible ground-closed trace. Its completeness is proved by fold-first membership saturation followed by a domain-weighted compiler, rather than inferred from semantic completeness.

1 Introduction

The companion paper develops the propositional core of nullivance logic (NPL): formulas valued in $[0,1]^2$ with independent truth and falsity channels, a threshold $\tau \in (0,1]$ inducing

*Part III: Finite-Domain Quantified Logic. Collection revision DR0149-r3 (9 November 2026). Mathematical content is retained from the August 2026 paper; this edition updates its presentation and artifact references. Companion to *Nullivance Propositional Logic: Threshold-Signed Consequence on the Unit Square with a Machine-Checked Completeness Theorem*. Results marked [L] are verified sorry-free in the Lean 4 artifact described in Appendix A. Related-work positioning is backed by a dated, source-by-source literature comparison; no priority claim is made for this collection edition.

†Contact: `lamtung0481@gmail.com`.

four signs, an exact projection onto the four-valued matrix **4**, and a four-signed analytic tableau calculus with a machine-checked completeness theorem. This paper asks the next question: what does a *quantified* NPL look like, and can its metatheory be machine-checked to the same standard?

We deliberately restrict to **finite nonempty domains**. The restriction is not cosmetic: the exact-projection theorem — the load-bearing bridge between the continuous semantics and the finite matrix — uses that finite infima and suprema are attained, and it can fail over infinite domains when a supremum is approached but not attained (Remark 3.9). Finite domains keep the entire two-layer architecture of NPL intact, and they are the natural first-order setting for the intended applications of a threshold logic (finite data, finite scenarios). Equality is *crisp*: $x{=}y$ evaluates to **T** or **F** by identity of domain elements, never to a glut or a gap.

Contributions. All results are verified sorry-free in Lean 4 unless explicitly marked otherwise; [L] markers name the verifying declarations.

1. *Finite-domain quantified NPL* (Section 3): function-free syntax with predicate atoms, crisp equality, the NPL connectives including $\oplus$, and quantifiers evaluated by finite channel-wise extrema; quantifier duality under channel-swap negation; and the exact threshold projection for the quantified language.

2. *A catalogue of machine-checked refutations* (Section 5): the naive signed tableau is incomplete (crisp reflexivity is valid but underivable); the bare grounding bridge to propositional NPL is unsound as a completeness route; and a natural dispatcher shortcut for the completeness proof fails on a two-element counterexample. Each failure shaped the final calculus.

3. *The core extensional tableau and its exact semantic characterization* (Sections 6 and 7): the headline theorem states that a finite branch closes in the macro-free core calculus *iff* it has no finite model, hence derivability coincides with finite semantic consequence. The proof introduces a domain-weighted formula measure under which every rule of the calculus — including the $(n{+}1)$-ary quantifier rules — strictly decreases the branch weight, and closes the literal stage by a canonical model read off the branch through an injective ground-atom encoding.

4. *The constrained-grounding corollary* (Section 8): closure of the grounded propositional branch augmented by *rigid constraints* (fixing the identity and equality atoms) yields core closure. Unlike the first completeness theorem of the artifact, no propositional-simulation macro rule appears anywhere: the propositional derivation is consumed through its soundness and the core closure is rebuilt semantically.

5. *A documented failure and conservative repair* (Section 9): the derivation-by-derivation simulation program (replay traces with residual fold tails) is developed to the point where its exact obstruction is isolated — the binary propositional decomposition cascade cannot be reassembled into the core's $(n{+}1)$-ary quantifier rules — and the weighted-measure proof is shown to dissolve the obstruction. A minimal admissible two-element fold cascade now refutes the proposed universal replay-certificate bridge itself, while leaving the semantic bridge intact. A separate certificate selects residual folds by membership; its projection to the core calculus and its repair of the minimal cascade are machine-checked. A fold-first saturation theorem and a domain-weighted flat compiler prove completeness of that new certificate for every admissible ground-closed trace.

Isn't this just propositional logic? A reader may object that finite-domain many-valued logic reduces routinely to propositional logic: ground out the quantifiers as finite folds and apply the companion paper's completeness theorem. Three machine-checked facts say the reduction is not routine. First, the naive lifted tableau is incomplete — crisp reflexivity is valid and underivable (Proposition 5.1). Second, the *bare* grounding bridge is refuted: propositional consequence of the grounded branch does not track finite-domain consequence, because propositional valuations may assign the identity and equality atoms values no finite-domain model induces (Proposition 5.3); repairing this needs the rigid constraints of Definition 4.3. Third, even with rigid constraints, the repaired route initially proved completeness only for a calculus with a propositional *simulation macro* — a single rule quoting the entire propositional tableau — which a proof-theoretic reading wants eliminated; eliminating it is exactly the content of Theorems 7.1 and 8.1, and the documented failure of the direct macro-elimination attempt (Section 9) shows the elimination has real content.

Method. The development follows the project's rigor rules: every claim is labeled, refutation is attempted before proof, and the Lean artifact is the ground truth. The manuscript numbering is presentational; the stable numbering and the full proofs live in the repository documentation.

2 Preliminaries: propositional NPL

We summarize what is needed from the companion paper. The propositional language has atoms and $\neg, \wedge, \vee, \oplus$. The matrix $\mathbf{4}$ is $\{0,1\}^2$ with corners $\mathbf{T} = (1,0)$, $\mathbf{F} = (0,1)$, $\mathbf{B} = (1,1)$, $\mathbf{N} = (0,0)$; negation swaps the two bits, $\wedge$ acts as $(\min, \max)$, $\vee$ as $(\max, \min)$, and $\oplus$ as $(\min, \min)$ (bilattice consensus [8, 7, 1]). The $\{\neg, \wedge, \vee\}$-fragment is Belnap–Dunn first-degree entailment [2, 6]. Four signs read the bits: T^+ (t-bit is 1), T^- (t-bit is 0), F^+, F^- likewise on the f-bit. A four-signed analytic tableau calculus (sixteen decomposition rules, two closure conditions) is sound and complete for signed consequence over $\mathbf{4}$, and — by the exact projection — over the continuous semantics on $[0,1]^2$; signed tableaux over sets-of-values signs are a standard paradigm [9]. All of this is machine-checked in the same artifact this paper extends.

3 Finite-domain quantified NPL

Definition 3.1 (Signature and syntax)**.** Fix countable stocks of variables and predicate symbols and a signature Σ assigning each predicate symbol P one arity $\mathrm{ar}_\Sigma(P) \in \mathbb{N}$. Quantified formulas are generated by

$$\varphi \ ::= \ P(x_0, \ldots, x_k) \ \mid \ x = y \ \mid \ \neg\varphi \ \mid \ \varphi \wedge \varphi \ \mid \ \varphi \vee \varphi \ \mid \ \varphi \oplus \varphi \ \mid \ \forall x\, \varphi \ \mid \ \exists x\, \varphi.$$

A formula is Σ-well-formed iff every occurrence $P(x_1, \ldots, x_k)$ has $k = \mathrm{ar}_\Sigma(P)$; the condition is inherited structurally by compound formulas. The artifact retains a total raw list syntax underneath this well-formed language. [L: `FiniteFO.QSignature`, `FiniteFO.QFormula`, `QFormula.WellFormed`]

Definition 3.2 (Models and assignments)**.** For a fixed $n \in \mathbb{N}$, the domain is $\mathrm{Fin}(n{+}1)$ — finite and nonempty by construction. A Σ-*model* M assigns to each predicate symbol P a map from tuples in $\mathrm{Fin}(n{+}1)^{\mathrm{ar}_\Sigma(P)}$ to $\mathbf{4}$. Off-arity arguments are therefore absent from the model type. The artifact retains a separate total raw list model for the tableau engine and connects the two model classes by extension and restriction (Proposition 3.5). An *assignment* ρ maps variables to domain elements; $\rho[x \mapsto d]$ updates it. [L: `FiniteFO.QSigModel`, `FiniteFO.QModel`, `FiniteFO.Assignment`]

Definition 3.3 (Evaluation)**.** Evaluation extends the propositional **4** clauses by crisp equality and finite channel-wise quantifiers:

$$V(x{=}y) = \begin{cases} \mathbf{T} & \rho(x) = \rho(y), \\ \mathbf{F} & \text{otherwise.} \end{cases}$$

$$V(\forall x\, \varphi) = \Big(\bigwedge_d t_d,\ \bigvee_d f_d\Big),$$
$$V(\exists x\, \varphi) = \Big(\bigvee_d t_d,\ \bigwedge_d f_d\Big).$$

where (t_d, f_d) is the value of φ under $\rho[x{\mapsto}d]$ and the extrema range over the finite domain. Signed satisfaction reads one bit of the value, exactly as in the propositional case. [L: `FiniteFO.QSigModel.eval`, `FiniteFO.qeval`, `forallV4`, `existsV4`]

Proposition 3.4 (Off-arity irrelevance)**.** *If two models agree on every predicate tuple whose length is declared by Σ, then they give the same value to every Σ-well-formed formula under every assignment.*

Proof. Structural induction on the formula. At a predicate atom, well-formedness makes the model-agreement hypothesis applicable; equality does not consult predicate data. The unary and binary cases follow from the induction hypotheses and the corresponding deterministic **4** operations. In a quantified case, apply the induction hypothesis at every updated assignment and substitute the pointwise equalities into the finite extrema. The hypothesis cannot be dropped: models may agree on all declared unary tuples and disagree on the malformed atom $P()$. [L: `FiniteFO.qeval_eq_of_agreeOn`] □

Proposition 3.5 (Raw/signature semantic bridge)**.** *Every Σ-model has a canonical total raw extension, obtained by assigning an arbitrary fixed value to off-arity lists. Every raw model restricts to a Σ-model. Restriction after extension recovers the original Σ-model, while extension after restriction agrees with the raw model on every declared-arity tuple. Consequently, for well-formed $\Gamma, s\varphi$, semantic consequence over Σ-models is equivalent to semantic consequence over raw models.*
[L: `FiniteFO.QSigModel.toRaw_restrict`, `FiniteFO.QModel.restrict_toRaw_agreeOn`, `FiniteFO.qconsequence4Sig_iff_qconsequence4`]

Proof. For a Σ-model, convert an arity-indexed tuple to its finite list and back; finite-function extensionality and the list `ofFn/get` identities recover the tuple. For a raw model, a declared-arity list is recovered after conversion to the induced tuple, so the extension of its restriction agrees on admitted tuples. For the consequence equivalence, transfer branch and conclusion satisfaction along this agreement using Proposition 3.4. In the reverse direction, apply raw consequence directly to the canonical extension of an arbitrary Σ-model. □

Definition 3.6 (Branches; semantic consequence)**.** A *signed branch member* is a triple $s(\rho, \varphi)$ of a sign, an assignment, and a formula — each member carries its *own* assignment. A *branch* is a finite list of members; a model *satisfies* a branch iff it satisfies every member. *Semantic consequence* is domain-indexed: $\Gamma \models_{Q,n} s\varphi$ iff every model over $\mathrm{Fin}(n{+}1)$ that satisfies every member of Γ satisfies $s\varphi$.
[L: `FiniteFO.QSigned`, `qsatSigned`, `qsatBranch`, `FiniteFO.QSigModel.satSigned`, `FiniteFO.QSigModel.satBranch`, `QConsequence4Sig`]

Proposition 3.7 (Quantifier duality)**.** *$\neg\forall x\,\varphi$ and $\exists x\,\neg\varphi$ have the same value, as do $\neg\exists x\,\varphi$ and $\forall x\,\neg\varphi$: channel-swap negation exchanges the two finite extrema. The nonempty domain is used; over an empty domain the statement would depend on conventions for vacuous quantifiers. [L: `FiniteFO.qeval_neg_all`, `qeval_neg_ex`]*

Theorem 3.8 (Finite exact projection)**.** *Interpret predicates by continuous truth-objects in $[0,1]^2$ and evaluate quantifiers by finite channel-wise infima/suprema. For every threshold $0 < \tau \leq 1$, thresholding commutes with evaluation: $\pi_\tau(V_M(\varphi)) = V_{\pi_\tau(M)}(\varphi)$, where $\pi_\tau(M)$ projects the predicate interpretations pointwise. Hence the finite-domain metatheory, like the propositional one, can be developed on* **4** *and lifted. [L: `FiniteFO.finite_exact_projection`]*

Remark 3.9 (Why finite domains)**.** Over an infinite domain the existential/supremum direction of Theorem 3.8 can fail when the supremum is not attained: every instance can fall below the threshold while the supremum reaches it. The finite, nonempty domain is therefore a load-bearing design choice, not a convenience. (Recorded as a refutation attempt in the development.)

4 Grounding

The finite domain makes every quantifier a finite fold, which suggests translating quantified branches into propositional ones.

Definition 4.1 (Ground atoms and the ground translation)**.** *Ground atoms* are: two fixed identity atoms $\top_g$ and $\bot_g$ (fold units), a ground predicate atom for each predicate symbol and argument list, and a ground equality atom for each pair of domain elements. Ground atoms are encoded injectively into propositional atoms. The translation gr_ρ maps a quantified formula under an assignment to a propositional formula: predicate and equality atoms to their ground atoms; connectives homomorphically; $\forall x\,\varphi$ to the finite conjunction fold of $\{\mathrm{gr}_{\rho[x\mapsto d]}(\varphi)\}_d$ (with unit $\top_g$) and $\exists$ to the disjunction fold (with unit $\bot_g$). [L: `FiniteFO.GroundAtom`, `groundAtomCode`, `FiniteFO.ground`, `foldConj`, `foldDisj`]

Lemma 4.2 (Grounding truth lemma)**.** *For every model M there is a propositional valuation v_M (reading ground atoms through M, with $\top_g \mapsto \mathbf{T}$, $\bot_g \mapsto \mathbf{F}$, and equality atoms crisp) such that the propositional value of $\mathrm{gr}_\rho(\varphi)$ under v_M equals the quantified value of φ under M, ρ. [L: `FiniteFO.ground_truth`, `groundVal`]*

Definition 4.3 (Rigid constraints)**.** The *rigid constraints* form the finite propositional branch containing T^+, F^- for $\top_g$; T^-, F^+ for $\bot_g$; and for every pair (a, b) of domain elements, T^+, F^- (respectively T^-, F^+) for the ground equality atom of (a, b) when $a = b$ (respectively $a \neq b$). They pin exactly the part of a ground valuation that finite-domain models do not choose freely; ground predicate atoms remain unconstrained. [L: `FiniteFO.rigidGroundConstraints`]

5 Calculi, and what fails

The propositional decomposition rules lift to signed quantified branch members, acting on the member's own assignment. For self-containedness we reproduce the sixteen propositional rows (from the companion paper; "A, B" adds both members to the branch, "$A \mid B$" splits it):

	T^+	T^-	F^+	F^-
$\neg\varphi$	$F^+\varphi$	$F^-\varphi$	$T^+\varphi$	$T^-\varphi$
$\varphi \wedge \psi$	$T^+\varphi, T^+\psi$	$T^-\varphi \mid T^-\psi$	$F^+\varphi \mid F^+\psi$	$F^-\varphi, F^-\psi$
$\varphi \vee \psi$	$T^+\varphi \mid T^+\psi$	$T^-\varphi, T^-\psi$	$F^+\varphi, F^+\psi$	$F^-\varphi \mid F^-\psi$
$\varphi \oplus \psi$	$T^+\varphi, T^+\psi$	$T^-\varphi \mid T^-\psi$	$F^+\varphi, F^+\psi$	$F^-\varphi \mid F^-\psi$

The eight finite quantifier rules come in two shapes, determined by which channel bit the sign reads of the finite extrema:

block (add all $n{+}1$ instances, non-branching)	$T^+\forall,\ F^-\forall,\ T^-\exists,\ F^+\exists$
per-element (one closed child per domain element)	$T^-\forall,\ F^+\forall,\ T^+\exists,\ F^-\exists$

Closure, in the naive calculus, is an identical opposite-signed pair (same assignment, same formula). [L: `FiniteFO.QCloses`]

Proposition 5.1 (The naive calculus is incomplete)**.** *Crisp reflexivity $T^+(x{=}x)$ is valid over every finite domain, but the empty branch does not derive it: no decomposition rule applies to an equality atom and no closure pair exists. Machine-checked countermodel to the completeness conjecture for `QCloses`. [L: `FiniteFO.qeqRefl0_valid`, `qeqRefl0_not_derivable`, `qcompleteness_current_refuted`]*

Definition 5.2 (Equality-completed calculus)**.** Add four crisp-equality closure clauses: a branch closes if it contains $T^-(x{=}y)$ or $F^+(x{=}y)$ under an assignment with $\rho(x) = \rho(y)$, or $T^+(x{=}y)$ or $F^-(x{=}y)$ with $\rho(x) \neq \rho(y)$. [L: `FiniteFO.QClosesEq`]

Proposition 5.3 (The bare grounding bridge fails)**.** *Grounding alone does not reduce finite-domain consequence to propositional consequence: $T^+(x{=}x)$ is a finite-domain validity, but the grounded sequent is* not *a propositional consequence — a propositional valuation may assign the ground equality atom of $(0, 0)$ any value, while every finite-domain model forces it to* **T**. *Machine-checked countermodel; the rigid constraints of Definition 4.3 repair exactly this gap. [L: `FiniteFO.qeqRefl0_ground_not_consequence4`, `qeqRefl0_ground_branch_not_closes`]*

Example 5.4 (Why equality clauses are not enough)**.** Let $\rho(x) = \sigma(y) = d$ and consider the two-member branch $\{T^+(\rho,\ P\,x),\ T^-(\sigma,\ P\,y)\}$. Every model gives both members the same ground predicate value at $(P, [d])$, so the branch is unsatisfiable; but the members are not an identical pair (different formulas, different assignments), no rule decomposes literals, and no equality clause applies. Closure must be allowed on *equal groundings*, not merely on identical members.

6 The core extensional tableau

Definition 6.1 (Core calculus)**.** The *core extensional finite-domain tableau* extends the equality-completed calculus of Definition 5.2 by two *ground closure clauses*: a branch closes if it contains members $T^+(\rho, \varphi)$ and $T^-(\sigma, \psi)$ with $\mathrm{gr}_\rho(\varphi) = \mathrm{gr}_\sigma(\psi)$, or F^+/F^- likewise. All sixteen propositional decomposition rules and all eight finite quantifier rules are retained. The calculus contains *no* propositional simulation macro. Derivability: $\Gamma \vdash_{\mathrm{core}} s\varphi$ iff the branch $\{\overline{s}\varphi\} \cup \Gamma$ closes, where $\overline{s}$ is the opposite sign. [L: `FiniteFO.QClosesExtCore`, `QDerivesExtCore`]

Proposition 6.2 (Soundness)**.** *If a branch closes in the core calculus, no finite model satisfies it; hence $\Gamma \vdash_{\mathrm{core}} s\varphi$ implies $\Gamma \models_{Q,n} s\varphi$. [L: `FiniteFO.QClosesExtCore.unsat`]*

Proof sketch. Induction on the closure derivation. Each closure condition is unsatisfiable at its own members: an identical pair by sign complementarity; an equality clause by crispness of $=$; a ground-closure pair because Lemma 4.2 gives both members the same value under any model, and opposite signs of one value cannot both hold. Each decomposition rule preserves satisfiability downward into (one of) its children by the corresponding local satisfaction equivalence — for the quantifier rules, the signed reading of the finite extrema. □

7 The completeness theorem

Theorem 7.1 (Semantic completeness of the core calculus)**.** *Fix* Σ*. Let* B *be a finite* Σ*-well-formed branch of signed quantified members over domain size* $n{+}1$*. If no model with domain* $\mathrm{Fin}(n{+}1)$ *satisfies* B*, then* B *closes in the core calculus. With Proposition 6.2, closure is* exactly *finite unsatisfiability. For every well-formed* Γ *and* $s\varphi$*,*

$$\Gamma \vdash_{\mathrm{core}} s\varphi \quad \Longleftrightarrow \quad \Gamma \models_{Q,n} s\varphi.$$

The artifact proves the same equivalence first for its stronger raw list syntax, proves the raw/signature semantic equivalence of Proposition 3.5, and composes the two equivalences.
[L: `FiniteFO.QClosesExtCore.complete_of_unsat`*,*
`qclosesExtCore_iff_unsat`*,* `QDerivesExtCore.complete`*,*
`qDerivesExtCore_iff_qconsequence4Sig`*]*

Proof (engine; full details in the artifact). Step 1: the domain-weighted measure. Define qsize on quantified formulas by qsize $= 1$ on predicate and equality atoms, $+1$ for negation, $+1$ over the sum for binary connectives, and

$$\mathrm{qsize}(\forall x\,\varphi) \;=\; \mathrm{qsize}(\exists x\,\varphi) \;=\; (n{+}1)\cdot \mathrm{qsize}(\varphi) + 1,$$

extended to branches by summation. Every rule of Definition 6.1 strictly decreases the weight of the undecomposed segment: unary and binary rules drop at least the head's outermost symbol; a block quantifier rule replaces weight $(n{+}1)w + 1$ by the $n{+}1$ instances of weight w each; a per-element rule replaces it by a single instance of weight $w \le (n{+}1)w$. The case analysis is exhaustive over the eight formula constructors and four signs.

Step 2: the engine. By strong induction on the weight bound, a branch split as *todo* + *lits* (with *lits* literals) that no finite model satisfies closes in the core: a literal head moves to *lits*; a compound head is decomposed by its rule, unsatisfiability transfers to each child by the corresponding local satisfaction equivalence (the signed quantifier equivalences and the propositional local-soundness table), the induction hypothesis applies at strictly smaller weight, and monotonicity of closure under branch extension [L: `QClosesExtCore.mono`] restores the full branch. Because the core's quantifier rules are natively $(n{+}1)$-ary, no intermediate "partially decomposed fold" state ever arises (contrast Section 9).

Step 3: the literal stage. For a branch of literals: if an equality literal contradicts its crisp value, an equality clause closes; if two literals carry T^+/T^- (or F^+/F^-) with equal groundings, a ground closure clause closes. Otherwise build the *canonical model*: the truth (falsity) bit of each ground predicate atom is set iff some T^+- (F^+-) literal grounds to it, read through the injective ground-atom encoding. Positive literals are satisfied by their own witness; negative literals because the opposite witness would be a ground closure pair, excluded by assumption; equality literals by crispness, their four unsatisfiable sign/value combinations being exactly the equality clauses. This contradicts unsatisfiability, so a closure option applied. □

Remark 7.2 (What the refutations bought)**.** Three hypotheses of Theorem 7.1 carry machine-checked failure witnesses for their removal: without the ground closure clauses, Example 5.4 is unsatisfiable but open; without the $(n{+}1)$-weighting, decomposing a block quantifier *increases* the unweighted measure and the induction has no footing; without any one of the four equality clauses, the corresponding literal configuration is unsatisfiable but unmodelable and unclosable.

8 Constrained grounding reaches the core

The artifact's first finite-domain completeness theorem ran through the propositional calculus: if the grounded branch together with the rigid constraints closes propositionally, an *extensional*

macro calculus closes the quantified branch by a single simulation rule. That macro rule is exactly what a proof-theoretic reading wants to eliminate.

Theorem 8.1 (Constrained grounding, macro-free)**.** *For every finite branch B: if the propositional branch* $\mathrm{Rigid}_n + \mathrm{gr}(B)$ *closes in the propositional tableau calculus, then B closes in the core calculus of Definition 6.1. No simulation macro occurs: the propositional derivation is consumed through propositional soundness, and the core closure is rebuilt by Theorem 7.1. Concretely: a finite model of B would induce, by Lemma 4.2 and the rigid values, a propositional valuation satisfying* $\mathrm{Rigid}_n + \mathrm{gr}(B)$*, contradicting propositional soundness; hence B is finitely unsatisfiable and Theorem 7.1 applies. [L:* `FiniteFO.groundBranch_closes_to_core`*,* `satBranch_groundVal_rigid`*]*

Conversely, grounding is sound: the grounded branch of a satisfiable quantified branch is satisfiable under the induced valuation (Lemma 4.2), so Theorem 8.1 composes with the propositional completeness theorem of the companion paper into a round-trip between the two levels.

9 The simulation program: failure and repair

An earlier phase of the project attempted Theorem 8.1 by *simulation*: induction on the propositional closure derivation, maintaining a replay trace whose items include quantified members, rigid atoms, and *residual fold tails* — the partially decomposed remains of a grounded quantifier, which are not groundings of any single quantified formula. The layer produced a complete, machine-verified dispatch analysis for propositional closure pairs (sixteen source combinations per closure sign), a recursive "tail consumer" closing any quantified formula against a fold tail it grounds into, and two further machine-checked refutations of shortcut routes.

The program then hit a precise wall: when the propositional derivation decomposes the grounding of a *per-element-sign* quantifier ($T^-\forall$ and its mirrors), the binary conjunction cascade produces a right child asserting the pooled fold of the remaining instances in one branch, while the core rule needs one closed child per domain element; the pooled induction hypothesis cannot be redistributed, and depth-preserving inversion does not make the induction well-founded. The weighted-measure engine of Theorem 7.1 dissolves the problem at the root: the core's quantifier rules are $(n{+}1)$-ary, so the binary cascade never appears. We record the dead end because the failure analysis, not luck, produced the measure; the simulation layer remains in the artifact as a verified study.

Definition 9.1 (Replay traces and the two certificate predicates)**.** Let $u = (\rho, \varphi)$ and write $q(S, u)$ for the quantified signed item (S, ρ, φ). A replay item is one of

$$q(S,u), \quad r(s), \quad C_S(L), \quad D_S(L), \quad C^q_S(U), \quad D^q_S(U),$$

where s is a ground signed formula, L is a list of ground formulas, and U is a list of assignment–formula pairs. A replay trace is a finite list of replay items. Its ground branch maps these six forms respectively to the grounding of $q(S, u)$, s, the signed conjunction/disjunction folds of L, and the signed folds of the groundings in U. Its quantified projection keeps $q(S, u)$ and expands $C^q_S(U), D^q_S(U)$ to the list of $q(S, u)$ for $u \in U$; the other forms contribute nothing.

A trace is *admissible* iff every rigid item belongs to the printed rigid constraint set, no unstructured $C_S(L)$ or $D_S(L)$ occurs, and the empty structured folds $C^q_{T^-}(\varnothing)$, $C^q_{F^+}(\varnothing)$, $D^q_{T^+}(\varnothing)$, and $D^q_{F^-}(\varnothing)$ do not occur. These are exactly the four false signed empty-fold cases.

The head-sensitive predicate $R(T)$ is the least proof predicate obtained by applying every closure and decomposition scheme of Definition 6.1 to a matching q-item in T, together with the

corresponding rigid-equality closure schemes, and the following fold schemes. For conjunction, $S \in \{T^+, F^-\}$ consumes a nonempty list as head plus residual tail, while $S \in \{T^-, F^+\}$ selects the list head; for disjunction the two sign pairs are exchanged. Empty folds are erased precisely in the four admissible empty cases. In all eight nonempty schemes the structured fold must be the *head* of T.

The membership-selecting predicate $R_\in(T)$ is the least predicate with $R(T) \Rightarrow R_\in(T)$ and the following eight schemes, where the displayed fold may occur anywhere in T:

fold	signs	recursive premise implying $R_\in(T)$
$C_S^q(u :: U) \in T$	$S \in \{T^+, F^-\}$	$R_\in(q(S,u) :: C_S^q(U) :: T)$
$C_S^q(U) \in T,\ u \in U$	$S \in \{T^-, F^+\}$	$R_\in(q(S,u) :: T)$
$D_S^q(u :: U) \in T$	$S \in \{T^-, F^+\}$	$R_\in(q(S,u) :: D_S^q(U) :: T)$
$D_S^q(U) \in T,\ u \in U$	$S \in \{T^+, F^-\}$	$R_\in(q(S,u) :: T)$

Each row abbreviates one constructor for each of its two signs. This definition is a proof-object layer only: induction over its nine constructor schemes proves $R_\in(T) \Rightarrow \mathrm{QClosesExtCore}(q(T))$.
[L: `FiniteFO.ReplayItem.Admissible`, `ReplayClosesCore`, `ReplayClosesCoreMem`, `ReplayClosesCoreMem.toCore`]

The old head-sensitive certificate conjecture also fails. Over a two-element domain, choose distinct ground predicate atoms p_0, p_1 and the admissible trace consisting of a positive structured conjunction tail for $[p_0, p_1]$ followed by the quantified source of $T^- p_1$. Its ground branch is

$$T^+(p_0 \wedge (p_1 \wedge \top_g)), \quad T^- p_1,$$

and closes by two positive-conjunction steps followed by atomic closure on p_1. Inverting a replay certificate first exposes p_0 ahead of the residual one-element fold. Since the exposed item is atomic, the residual fold is no longer at the head where the certificate's fold constructors require it, and the only possible atomic close would identify $p_0 = p_1$. Injectivity of the ground-atom encoding contradicts that equality. Thus admissibility plus propositional closure does not imply an existing replay certificate. This is a failure of the head-sensitive certificate representation, not of core soundness or completeness.
[L: `FiniteFO.admissible_ground_replay_bridge_refuted`]

The repair is deliberately a new proof-object type rather than a revision of the refuted claim. The membership-selecting certificate embeds every old certificate, but its structured-fold constructors may select a fold occurrence anywhere in the trace. All-item signs expose the head and residual tail; branching signs select any represented item. Induction on the nine new constructor schemes proves sound projection into the macro-free core calculus. On the exact counterexample above, two membership selections expose p_1 and an embedded atomic close finishes the certificate. Conversely, the earlier inadmissible empty-fold witness still has no new certificate: sound projection would close the empty quantified branch. Thus the extension repairs the observed defect without certifying the known false positive.
[L: `FiniteFO.ReplayClosesCoreMem.toCore`,
`FiniteFO.replayCascadeTrace_old_refuted_mem_verified`,
`FiniteFO.replayEmptyBadTailTrace_not_replayClosesCoreMem`]

We now prove the universal bridge for the repaired certificate. For a trace T and quantified worklist B, let its flat normal form consist of four signed atoms fixing the empty conjunction and disjunction folds, the grounding of B, and exactly the rigid items retained by T. A finite reach relation generated by the membership-selecting constructors first extends T until every member of its quantified projection occurs as a direct quantified item. This removes the global-choice obstruction: all branching alternatives may be materialized before compilation.

Theorem 9.2 (Universal membership replay bridge)**.** *Every admissible replay trace whose ground branch closes has a membership-selecting replay certificate.*

Proof. Fold-first saturation gives an admissible extension U containing the original trace and directly representing every member of its quantified projection. Ground closure is monotone under this extension. Any valuation satisfying the flat normal form of U satisfies its original ground trace: direct quantified and rigid items transfer immediately; structured folds transfer by induction over their represented finite lists, with the four fixed atoms discharging empty residual folds. Propositional soundness and completeness therefore close the flat normal form of U.

It remains to compile that closure into the old head-sensitive certificate over U. We prove a stronger statement by induction on an upper bound for the domain-weighted size of a worklist *todo*. A second list *lits* contains only predicate or crisp-equality literals, and every member of *todo* ++ *lits* is required to occur directly in U. With empty *todo*, atomic closure inversion exhausts the identity, quantified, and rigid sources and builds an old certificate. A literal at the head is moved to *lits*; the resulting permutation preserves satisfaction and reduces the worklist weight.

For negation and the three binary connectives, prepend the child item or items prescribed by the old replay constructor. The applicable **4** truth-table equation shows that satisfaction of each required child flat branch implies satisfaction of the parent flat branch. Parent unsatisfiability and propositional completeness consequently close the child branches. Their measures are smaller: the outer connective unit is removed, and a selecting branch also discards a subformula of positive size. Apply the induction hypothesis and the corresponding replay constructor.

For an all-instance quantifier rule, prepend its $(n+1)$-element instance block. Its weight is $(n+1)\mathrm{qsize}(\varphi)$, one below the parent's $(n+1)\mathrm{qsize}(\varphi)+1$, and satisfaction of all grounded instances implies satisfaction of the parent finite fold. For a selecting rule, repeat the argument for each $d \in \mathrm{Fin}(n{+}1)$ using membership of the selected grounding in the fold. Positivity of $n{+}1$ gives $\mathrm{qsize}(\varphi) \le (n+1)\mathrm{qsize}(\varphi) < (n+1)\mathrm{qsize}(\varphi)+1$; this includes $n=0$. These cases cover both quantifiers and all four signs. The resulting old certificate for U is transported backwards along the finite saturation path, yielding the required membership-selecting certificate for T. □

[L: `FiniteFO.ReplayTrace.flat_closes_to_replay`,
`FiniteFO.admissible_ground_replay_bridge_mem_verified`]

The semantic route alone would still be insufficient: it produces core closure rather than a value of the certificate type. The theorem above supplies the separate normalization and compilation argument. It does not resurrect the refuted old bridge, whose conclusion uses the strictly smaller head-sensitive certificate.

10 Related work

Finitely-valued first-order tableaux. Complete tableau calculi for arbitrary finite many-valued first-order logics are classical: Carnielli's systematization [5] proves abstract completeness, model existence, compactness and Löwenheim–Skolem theorems for tableaux with *distribution quantifiers*, and Hähnle's monograph [9] generalizes the sign language to sets-as-signs. We therefore claim no novelty for many-valued first-order tableau completeness as such, and NPL's four threshold signs are themselves sets-as-signs instances ($\{\mathbf{T},\mathbf{B}\}$, $\{\mathbf{F},\mathbf{N}\}$, $\{\mathbf{B},\mathbf{F}\}$, $\{\mathbf{T},\mathbf{N}\}$). The present calculus differs in format and scope: quantifiers range over a *fixed finite nonempty domain* and are decomposed by $(n{+}1)$-ary instance rules rather than through value distributions over arbitrary domains; the closure conditions include crisp-equality clauses and

ground-closure (extensional) clauses, neither of which is supplied by the generic per-formula calculi cited above; and the completeness proof is driven by the domain-weighted measure, which has no counterpart in the distribution-quantifier setting. Two delimitations are worth making explicit. Semantically, fixing the domain *in the judgment* is a design choice, not a lack: the intended applications quantify over known finite data, and the domain-indexed consequence relation (Definition 3.6) is the natural primitive there — the distribution-quantifier tradition answers a different question (consequence over all structures). Proof-theoretically, one may ask whether the core calculus is an instance of the sets-as-signs framework whose generic completeness argument applies off the shelf: the decomposition rules are, but the crisp-equality clauses and the ground-closure clauses act on *pairs of members across different assignments* through the grounding translation, which is outside the per-formula format of the generic argument; Example 5.4 is precisely a branch that every sets-as-signs closure notion keyed to identical signed formulas leaves open.

Quantified Belnap–Dunn logics. First-order FDE and its tableaux are textbook material [10, Ch. 22], on the classical foundations of [2, 6]. Recent work studies free quantification over four-valued and fuzzy bilattice-valued logics with dual-domain semantics for non-denoting terms [4]; that axis (free logic, unrestricted domains, no mechanized proof theory) is orthogonal to ours (total denotation, fixed finite domains, machine-checked exact characterization). The harmonization connective $\oplus$ in the object language and the threshold reading remain the NPL-specific propositional base, positioned against the bilattice literature in the companion paper.

Machine-checked completeness. Formalized metatheory exists for classical first-order proof systems, including an Isabelle/HOL coinductive framework instantiated for Gentzen and tableau variants of classical logic [3]. TableauxRocq gives a recent Rocq verification of soundness and certificate checking for a classical first-order tableau [11]; it does not claim a many-valued completeness result. Formalized paraconsistent metatheory also predates this work [12]. The contribution established here is the machine-checked completeness theorem for this specified many-valued calculus. An earlier search dated 2026-08-16 does not establish priority at the date of this revised collection; no claim of being the first is made.

11 Open problems

(1) *Infinite domains*: the projection argument fails at unattained suprema (Remark 3.9); a quantified NPL over infinite domains needs either attained-extrema semantics or a genuinely different bridge. Part I reports later exact procedures for the pure-equality fragment; these do not settle the unrestricted problem stated here. (2) *Function symbols and non-crisp equality*: the present language is function-free and its equality never takes glut or gap values; both restrictions are design choices worth revisiting. (3) *Complexity*: finite-domain consequence is decidable by finiteness of the model space; its exact complexity as a function of domain size and formula size is open here. (4) *Certificate complexity and normalization*: the repaired membership-selecting certificate is complete by Theorem 9.2, but the present proof materializes all quantified-projection members before compilation. Tight upper bounds on certificate growth, and a focused or zipper-style compiler that avoids unnecessary materialization, remain open.

A The Lean artifact

The results of this paper are verified in the `Nullivance` Lean 4 library, principally module `Nullivance.FiniteFO`, and share the artifact of the companion paper. **Availability.** The complete source artifact is supplied with this release, including citation/license metadata,

exact dependency locks, all three manuscripts, and verification scripts. Select the record labeled DR0149-r3 in the version family 10.5281/zenodo.18309848. The v0.7.1 edition remains a separate historical artifact. **Toolchain.** Lean 4 (`leanprover/lean4:v4.32.1`) and mathlib `v4.32.1` at commit `520045ab14e26149ee970e2e617ca04b09bde5d6`; run `scripts/Verify-ResearchPublication.ps1`. **Coverage policy.** The build is `sorry`-free. The headline theorems are stated directly against `QClosesExtCore`; replay-certificate results use the separately defined proof-object predicates of Definition 9.1 and project into that core. Axiom audit: every named declaration depends only on `propext`, `Classical.choice`, `Quot.sound`. **Key declarations.**
Semantics: `QSigModel`, `qeval`, `qsatBranch`.
Projection: `finite_exact_projection`.
Refutations: `qcompleteness_current_refuted`, `qeqRefl0_not_derivable`.
Calculus: `QClosesExtCore`; soundness: `QClosesExtCore.unsat`.
Completeness engine: `qsize`, `qweightB`, `qclosesCore_todo`, `qclosesCore_lits`.
Headline: `QClosesExtCore.complete_of_unsat`,
`qDerivesExtCore_iff_qconsequence4Sig`.
Grounding: `ground_truth`, `groundBranch_closes_to_core`.
Replay repair: `ReplayClosesCoreMem`, `ReplayClosesCoreMem.toCore`,
`replayCascadeTrace_old_refuted_mem_verified`.
Replay completeness: `ReplayTrace.flat_closes_to_replay`,
`admissible_ground_replay_bridge_mem_verified`.

References

[1] Ofer Arieli and Arnon Avron. Reasoning with logical bilattices. *Journal of Logic, Language and Information*, 5(1):25–63, 1996.

[2] Nuel D. Belnap, Jr. A useful four-valued logic. In J. Michael Dunn and George Epstein, editors, *Modern Uses of Multiple-Valued Logic*, pages 5–37. D. Reidel, Dordrecht, 1977.

[3] Jasmin Christian Blanchette, Andrei Popescu, and Dmitriy Traytel. Soundness and completeness proofs by coinductive methods. *Journal of Automated Reasoning*, 58(1):149–179, 2017.

[4] Libor Běhounek, Martina Daňková, and Antonín Dvořák. Free quantification in four-valued and fuzzy bilattice-valued logics. In *Integrated Uncertainty in Knowledge Modelling and Decision Making*, volume 14375 of *Lecture Notes in Computer Science*, pages 15–26. Springer, 2023.

[5] Walter A. Carnielli. Systematization of finite many-valued logics through the method of tableaux. *The Journal of Symbolic Logic*, 52(2):473–493, 1987.

[6] J. Michael Dunn. Intuitive semantics for first-degree entailments and "coupled trees". *Philosophical Studies*, 29(3):149–168, 1976.

[7] Melvin Fitting. Bilattices and the semantics of logic programming. *Journal of Logic Programming*, 11(2):91–116, 1991.

[8] Matthew L. Ginsberg. Multivalued logics: A uniform approach to reasoning in artificial intelligence. *Computational Intelligence*, 4(3):265–316, 1988.

[9] Reiner Hähnle. *Automated Deduction in Multiple-Valued Logics*, volume 10 of *International Series of Monographs on Computer Science*. Oxford University Press, Oxford, 1994.

[10] Graham Priest. *An Introduction to Non-Classical Logic: From If to Is*. Cambridge University Press, Cambridge, 2 edition, 2008.

[11] Johann Rosain and Julie Cailler. TableauxRocq: A deep embedding of free-variable tableaux in Rocq. In Ekaterina Komendantskaya and Tobias Nipkow, editors, *17th International Conference on Interactive Theorem Proving (ITP 2026)*, volume 382 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 12:1–12:22, Dagstuhl, Germany, 2026. Schloss Dagstuhl–Leibniz-Zentrum für Informatik.

[12] Jørgen Villadsen and Anders Schlichtkrull. Formalizing a paraconsistent logic in the Isabelle proof assistant. In *Transactions on Large-Scale Data- and Knowledge-Centered Systems XXXIV*, volume 10620 of *Lecture Notes in Computer Science*, pages 92–122. Springer, 2017.